

Responde en tu cuaderno o en un documento, con tus propias palabras:

1. ¿Qué es una API y qué significa que una API sea REST?

Una API (Application Programming Interface) es un conjunto de reglas y mecanismos que permite que diferentes aplicaciones se comuniquen entre sí e intercambien información. Una API REST es una API que utiliza los principios de REST (Representational State Transfer) y normalmente emplea métodos HTTP como GET, POST, PUT, PATCH y DELETE para consultar, crear, modificar o eliminar recursos.

2. Describe la diferencia entre cliente y servidor en una petición HTTP.

El cliente es el programa o dispositivo que realiza una petición al servidor, por ejemplo, Postman o un programa escrito en Python. El servidor recibe la petición, la procesa y devuelve una respuesta con información, un código de estado y encabezados HTTP.

Ejemplo:

*Cliente: Postman.

*Petición: `GET /posts/1`

*Servidor: JSONPlaceholder.

*Respuesta: información de la publicación solicitada.

Metodo	Operación CRUD	Para que se usa	Codigo de éxito típico
GET	Leer	Consultar u obtener información de un recurso.	200 OK
POST	Crear	Crear un nuevo recurso.	201 Created
PUT	Actualizar	Reemplazar o actualizar completamente un recurso.	200 OK
PATCH	Actualizar	Modificar parcialmente un recurso.	200 OK
DELETE	Eliminar	Eliminar un recurso	200 OK

CODIGO	SIGNIFICADO	EXPLICACION
200	Ok	La petición se realizó correctamente.
201	Created	Se creó correctamente un nuevo recurso.
204	No content	La petición fue exitosa, pero el servidor no devuelve contenido en la respuesta.
400	Bad request	La petición tiene un error y el servidor no puede procesarla correctamente.
401	Unauthorized	Se requiere autenticación o las credenciales proporcionadas no son válidas.
404	Not found	El recurso solicitado no existe o no fue encontrado.
500	Internal Server Error	Ocurrió un error interno en el servidor.

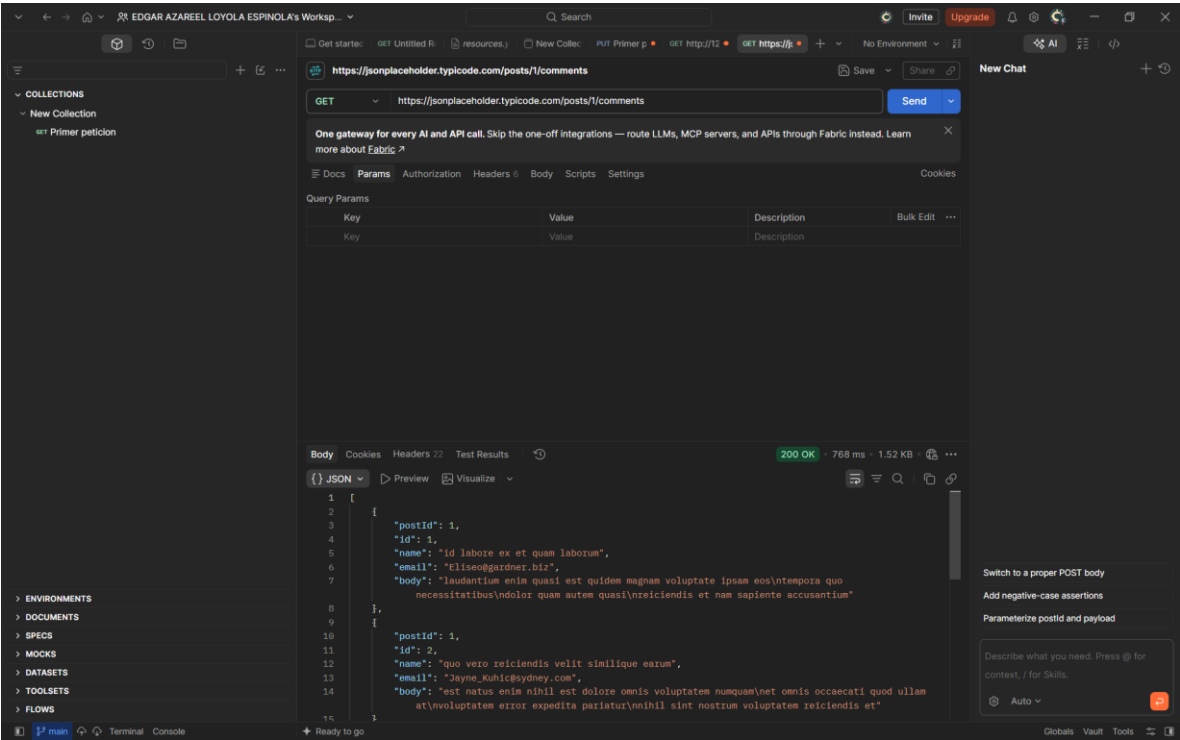
5. Objeto JSON de un dispositivo de red

```
{
  "nombre": "Router-Cisco-01",
  "direccion_ip": "192.168.1.1",
  "modelo": "Cisco 2911",
  "estado": "activo",
  "interfaces": [
    "GigabitEthernet0/0",
    "GigabitEthernet0/1",
    "Serial0/0/0"
  ]
}
```

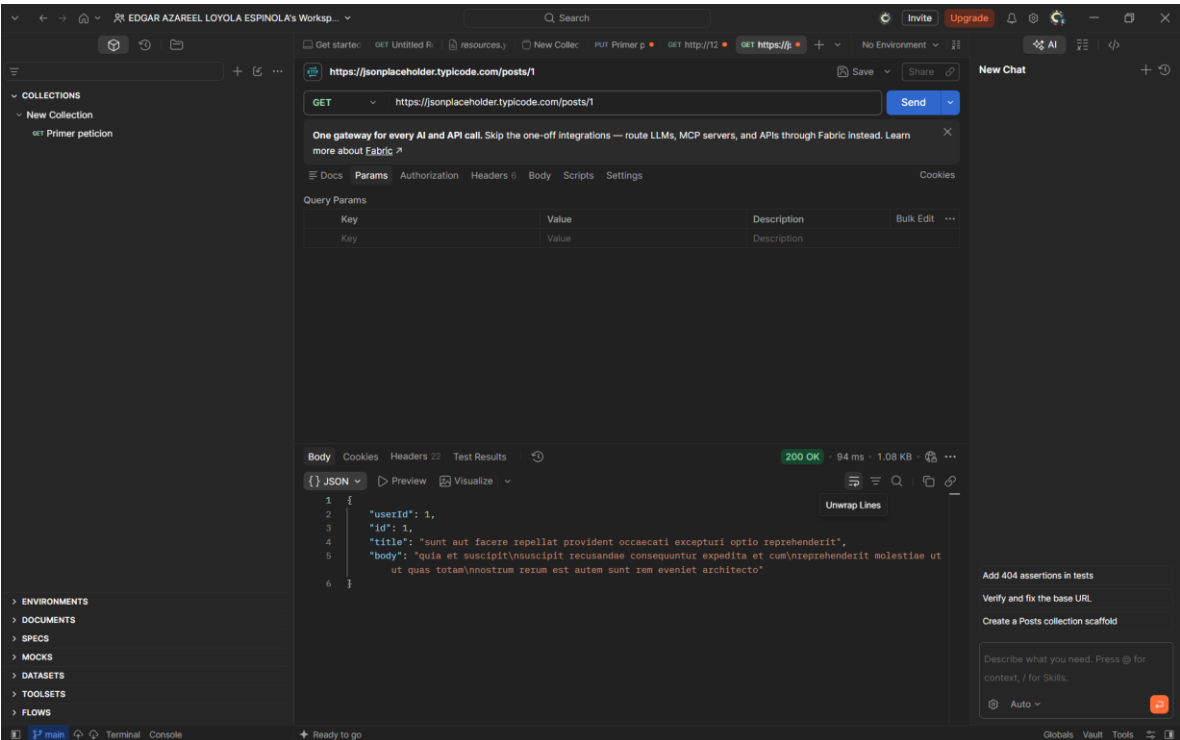
Elemento	Valor	Tipo de dato
Nombre	"Router-Cisco-01"	String / cadena
Dirección ip	"192.168.1.1"	String / cadena
Modelo	"Cisco 2911"	String / cadena
Estado	"activo"	String / cadena
Interfaces	[...]	Array / lista
Elemento de interfaces	"GigabitEthernet0/0"	String / cadena

4.Parte B. Consumo de la API con Postman (60 min)

B1



B2



The screenshot displays the Postman application interface. At the top, there's a search bar and navigation icons. The left sidebar shows a 'COLLECTIONS' panel with a 'New Collection' button and a 'GET Primer petition' item. The main workspace is divided into two sections: the top section shows the request details, and the bottom section shows the response details.

Request Details:

- Method:** GET
- URL:** https://jsonplaceholder.typicode.com/posts?userId=3
- Headers:** Params, Authorization, Headers, Body, Scripts, Settings
- Query Params:**

Key	Value	Description	Bulk Edit
userId	3		

Response Details:

- Status:** 200 OK
- Time:** 77 ms
- Size:** 1.86 KB
- Body:**

```
[
  {
    "userId": 3,
    "id": 21,
    "title": "asperiores ea ipsam voluptatibus modi minima quia sint",
    "body": "repellat aliquid praesentium dolorem quo\u00d2sed totam minus non itaque\u00d2nnihil labore molestiae sunt dolor eveniet hic recusandae veniam\u00d2tempora et tenetur expedita sunt"
  },
  {
    "userId": 3,
    "id": 22,
    "title": "dolor sint quo a velit explicabo quia nam",
    "body": "eos qui et ipsum ipsam suscipit aut\u00d2sed omnis non odio\u00d2nexpedita earum mollitia molestiae aut atque rem suscipit\u00d2nam impedit esse"
  }
]
```

Right Sidebar:

- New Chat:** A section for interacting with an AI assistant. It includes a prompt area with the text "Describe what you need. Press @ for context, / for Skills." and a "Send" button.

The screenshot displays the VS Code interface with a REST client extension. The main editor shows a REST client request to `https://jsonplaceholder.typicode.com/posts/1/comments` using the `GET` method. The response is a `200 OK` status with a JSON body containing two comment objects. The interface includes a sidebar with 'COLLECTIONS' and 'ENVIRONMENTS', a top bar with 'Invited' and 'Upgrade' buttons, and a bottom bar with 'Terminal' and 'Console' tabs.

Request:

```
GET https://jsonplaceholder.typicode.com/posts/1/comments
```

Response:

```
200 OK - 324 ms - 40.35 KB
{
  "posts": [
    {
      "postId": 1,
      "id": 1,
      "name": "id labore ex et quam laborum",
      "email": "Eliseo@gardner.biz",
      "body": "laudantium enim quasi est quidem magnam voluptate ipsam eos\ntempora quo necessitatibus\ndolor quam autem quasi\nreiciendis et nam sapiente accusantium"
    },
    {
      "postId": 1,
      "id": 2,
      "name": "quo vero reiciendis velit similique earum",
      "email": "Jayne_Kuhic@sydney.com",
      "body": "est natus enim nihil est dolore omnis voluptatem numquam\net omis occasanti quod ullam\nat\involutatem error expedita pariatur\nnihil sint nostrum voluptatem reiciendis et"
    }
  ]
}
```

B5

Postman interface showing a POST request to `https://jsonplaceholder.typicode.com/posts`. The response is **201 Created** with a status of 117 ms and 1.01 KB. The response body is JSON:

```
{
  "id": 181
}
```

The interface includes a sidebar with collections, environments, documents, specs, mocks, datasets, toolsets, and flows. The main panel shows the request details, including the URL, method (POST), and the response body. The bottom status bar indicates 'Ready to go'.

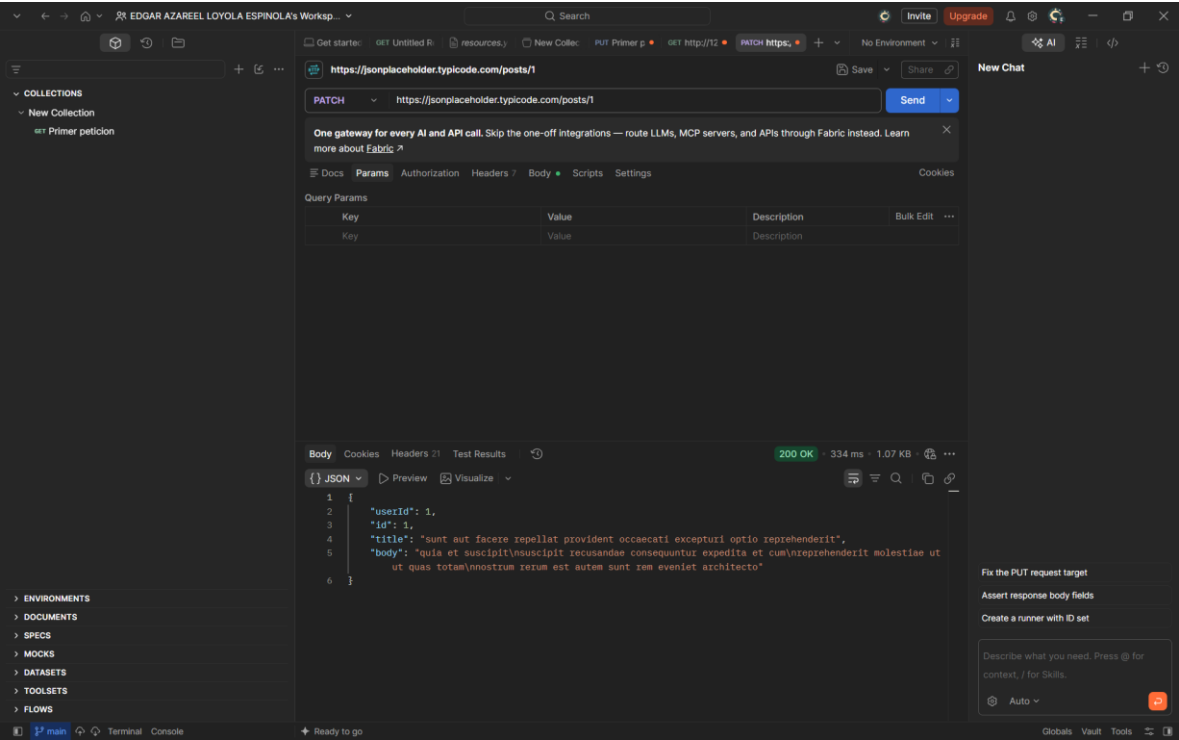
B6

Postman interface showing a PUT request to `https://jsonplaceholder.typicode.com/posts/1`. The response is **200 OK** with a status of 323 ms and 906 B. The response body is JSON:

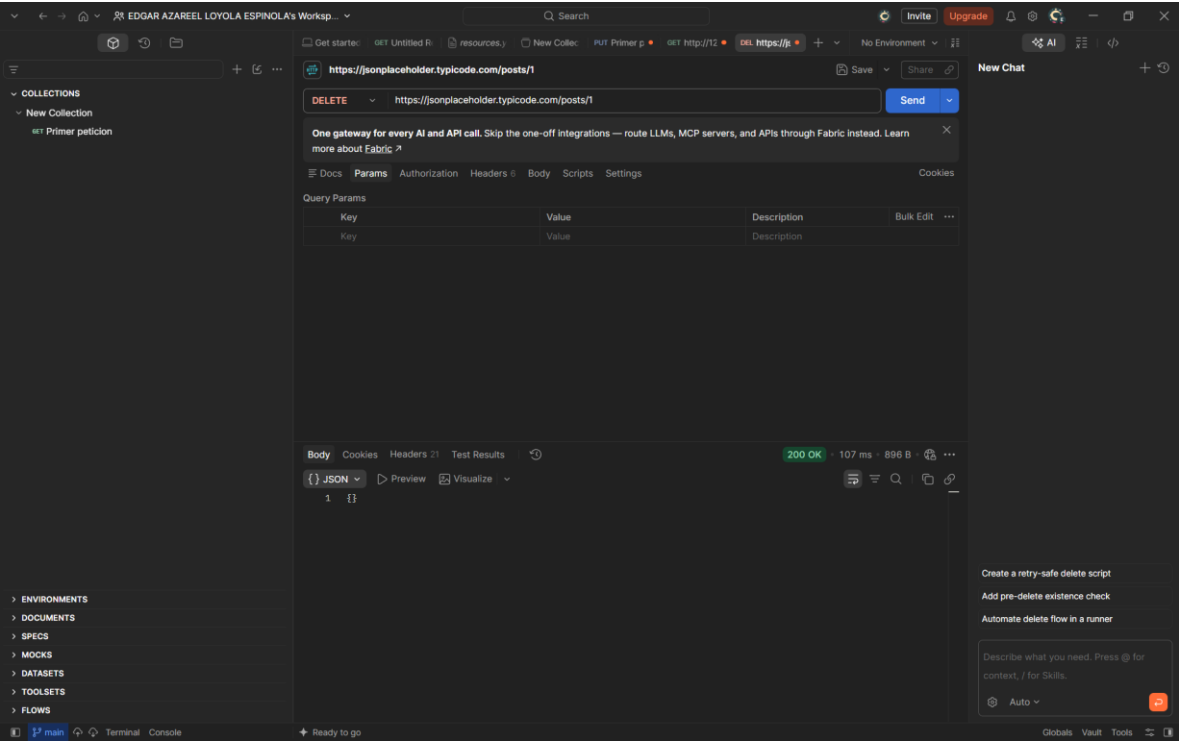
```
{
  "id": 1
}
```

The interface includes a sidebar with collections, environments, documents, specs, mocks, datasets, toolsets, and flows. The main panel shows the request details, including the URL, method (PUT), and the response body. The bottom status bar indicates 'Ready to go'.

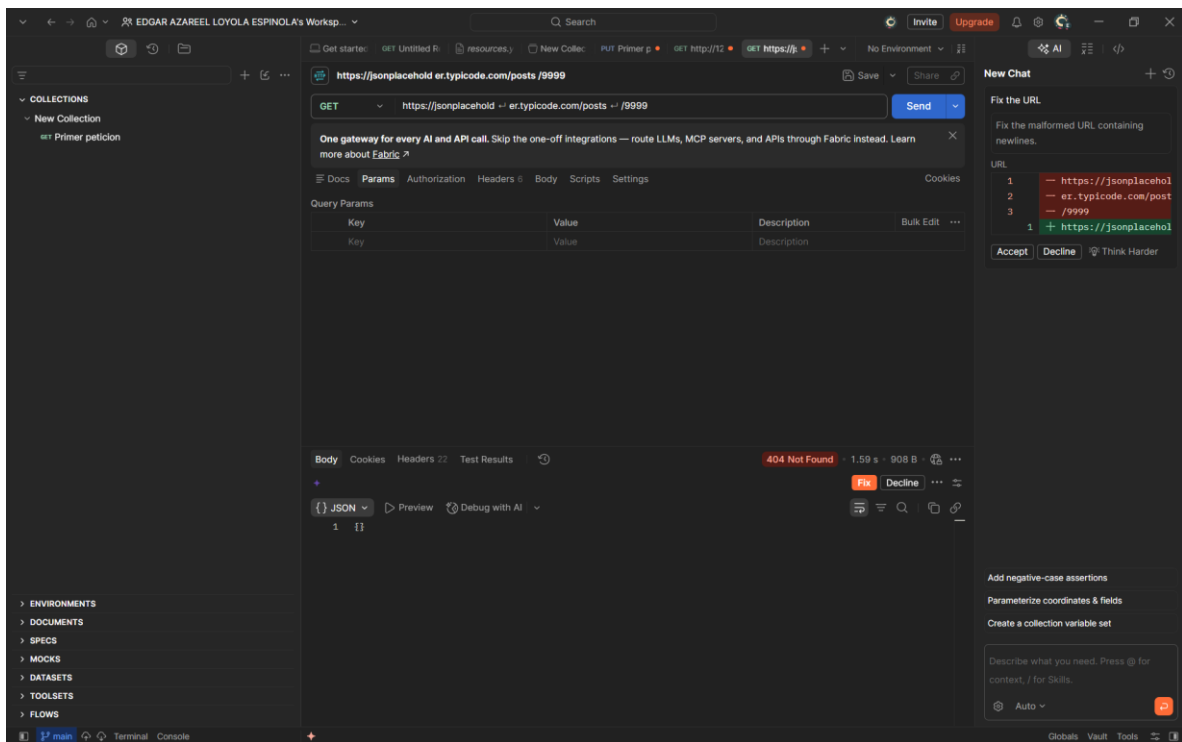
B7



B8

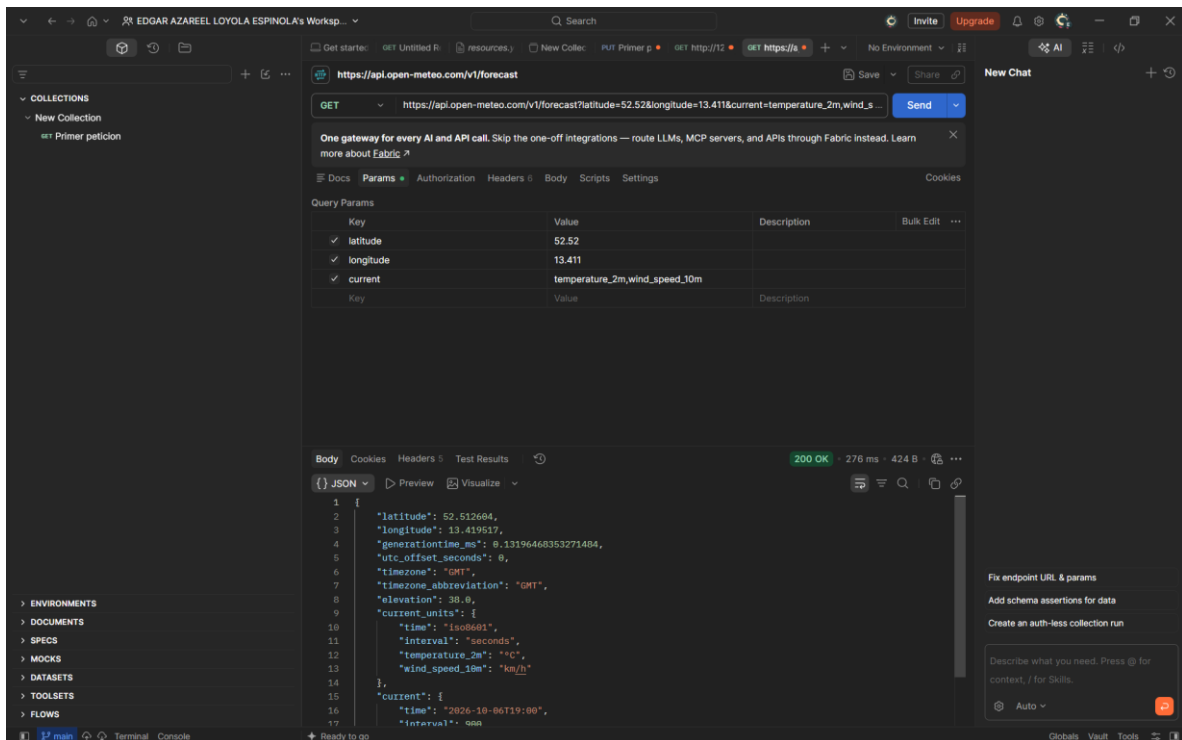


B9



No encontré lo que me pediste

B10



Preguntas de la Parte B

1. ¿Qué diferencia observaste entre PUT (B6) y PATCH (B7) en lo que se envía?

La diferencia principal es la cantidad de información que se envía. En **PUT** se envía el objeto completo para reemplazar o actualizar el recurso, mientras que en **PATCH** solamente se envían los campos que se desean modificar.

2. ¿Qué encabezado Content-Type regresa el servidor y qué indica?

El servidor regresa el encabezado:

Content-Type: application/json; charset=utf-8

Este encabezado indica que el contenido de la respuesta está en formato JSON y que utiliza la codificación UTF-8.

3. En B10, ¿qué parte de la URL es la ruta y cuál son los parámetros de consulta (query params)?

La ruta de B10 es /v1/forecast y los parámetros de consulta son latitude=20.59, longitude=-100.39 y current=temperature_2m,wind_speed_10m.

5. Parte C. Consumo de la API con Python

C1

```
import requests

BASE = "https://jsonplaceholder.typicode.com"

respuesta = requests.get(f"{BASE}/posts/1", timeout=10)

print("Código de estado:", respuesta.status_code)
print("Tipo de contenido:", respuesta.headers["Content-Type"])

publicacion = respuesta.json()

print("Título:", publicacion["title"])

print("Edgar Azareel Loyola Espinola")
```

```
C:\Users\Aza\Unidad-2-AZAREEL\actividad_rest>python C1_cliente_rest.py
Código de estado: 200
Tipo de contenido: application/json; charset=utf-8
Título: sunt aut facere repellat provident occaecati excepturi optio reprehenderit
Edgar Azareel Loyola Espinola
```


C2

```
import requests

BASE = "https://jsonplaceholder.typicode.com"

respuesta = requests.get(
    f"{BASE}/posts",
    params={"userId": 3},
    timeout=10
)

publicaciones = respuesta.json()

print("Total:", len(publicaciones))

for p in publicaciones:
    print(f"{p['id']}>3} | {p['title']}")
print("Edgar Azareel Loyola Espinola")
```

```
C:\Users\Aza\Unidad-2-AZAREEL\actividad_rest>python C2_cliente_rest.py
Total: 10
21 | asperiores ea ipsam voluptatibus modi minima quia sint
22 | dolor sint quo a velit explicabo quia nam
23 | maxime id vitae nihil numquam
24 | autem hic labore sunt dolores incidunt
25 | rem alias distinctio quo quis
26 | est et quae odit qui non
27 | quasi id et eos tenetur aut quo autem
28 | delectus ullam et corporis nulla voluptas sequi
29 | iusto eius quod necessitatibus culpa ea
30 | a quo magni similique perferendis
Edgar Azareel Loyola Espinola
```

C3

```

import requests

BASE = "https://jsonplaceholder.typicode.com"

# POST - Crear
nuevo = {
    "title": "Prueba de red",
    "body": "Contenido de ejemplo",
    "userId": 1
}

r = requests.post(
    f"{BASE}/posts",
    json=nuevo,
    timeout=10
)

print("POST:", r.status_code)
print("Respuesta:", r.json())

# PUT - Reemplazar
r = requests.put(
    f"{BASE}/posts/1",
    json={**nuevo, "id": 1},
    timeout=10
)

print("PUT:", r.status_code)
print("Respuesta:", r.json())

# DELETE - Eliminar
r = requests.delete(
    f"{BASE}/posts/1",
    timeout=10
)

print("DELETE:", r.status_code)
print("Edgar Azareel Loyola Espinola")

```

```

C:\Users\Aza\Unidad-2-AZAREEL\actividad_rest>python C3_cliente_rest.py
POST: 201
Respuesta: {'title': 'Prueba de red', 'body': 'Contenido de ejemplo', 'userId': 1, 'id': 101}
PUT: 200
Respuesta: {'title': 'Prueba de red', 'body': 'Contenido de ejemplo', 'userId': 1, 'id': 1}
DELETE: 200
Edgar Azareel Loyola Espinola

```

C4

```
import requests

BASE = "https://jsonplaceholder.typicode.com"

try:
    r = requests.get(
        f"{BASE}/posts/9999",
        timeout=10
    )

    r.raise_for_status()

    print(r.json())

except requests.exceptions.HTTPError as e:
    print("Error HTTP:", e)

except requests.exceptions.RequestException as e:
    print("Error de conexión:", e)
print("Edgar Azareel Loyola Espinola")
```

```
C:\Users\Aza\Unidad-2-AZAREEL\actividad_rest>python C4_cliente_rest.py
Error HTTP: 404 Client Error: Not Found for url: https://jsonplaceholder.typicode.com/posts/9999
Edgar Azareel Loyola Espinola
```

C5

```
import requests

def obtener_clima(latitud, longitud):
    url = "https://api.open-meteo.com/v1/forecast"

    parametros = {
        "latitude": latitud,
        "longitude": longitud,
        "current": "temperature_2m,wind_speed_10m"
    }

    r = requests.get(
        url,
        params=parametros,
        timeout=10
    )

    r.raise_for_status()

    return r.json()["current"]

# Tres ciudades
ciudades = {
    "Querétaro": (20.59, -100.39),
    "Ciudad de México": (19.43, -99.13),
    "Guanajuato": (21.02, -101.25)
}

print("CLIMA ACTUAL")
print("-" * 50)

for ciudad, coordenadas in ciudades.items():
    try:
        clima = obtener_clima(coordenadas[0], coordenadas[1])

        print(
            f"{ciudad}: "
            f"Temperatura = {clima['temperature_2m']} °C | "
            f"Viento = {clima['wind_speed_10m']} km/h"
        )

    except requests.exceptions.RequestException as e:
        print(f"{ciudad}: Error de conexión - {e}")

print("Edgar Azareel Loyola Espinola")
```

```
C:\Users\Aza\Unidad-2-AZAREEL\actividad_rest>python C5_cliente_rest.py  
CLIMA ACTUAL
```

```
-----  
Querétaro: Temperatura = 24.5 °C | Viento = 14.6 km/h  
Ciudad de México: Temperatura = 21.7 °C | Viento = 6.0 km/h  
Guanajuato: Temperatura = 22.4 °C | Viento = 15.9 km/h  
Edgar Azareel Loyola Espinola
```

```
C:\Users\Aza\Unidad-2-AZAREEL\actividad_rest>|
```